Experiment 4: Build API to Serve ML Model Predictions via Flask

Aim

To create a REST API using Flask that loads pre-trained ML models and serves predictions through HTTP endpoints.

Software Requirements

- Python 3.10+
- Flask
- scikit-learn
- pickle
- **Previously trained models from Experiment 3**

Theory

REST (Representational State Transfer) APIs allow different applications to communicate over HTTP. By exposing ML models through API endpoints, we enable any client (web, mobile, desktop) to make predictions. The API accepts input features as JSON, processes them through the loaded model, and returns predictions. This architecture separates the ML logic from the presentation layer, following microservices principles.

Procedure

Step 1: Create Project Directory

```
mkdir exp4_ml_api cd
exp4_ml_api
```

Step 2: Copy Model Files

Copy the model files from Experiment 3:

```
cp ../exp3_ml_training/*.pkl .
```

Step 3: Create and Activate Virtual Environment

```
python -m venv venv
```

Windows:

```
venv\Scripts\activate
```

Mac/Linux:

```
source venv/bin/activate
```

Step 4: Install Required Packages

```
pip install flask scikit-learn
```

Step 5: Create app.py

```
from flask import Flask, request, jsonify, render_template
import pickle
import numpy as np

app = Flask(__name__

)

# Load models at startup
print("Loading models...")
with open('iris_logistic_regression.pkl', 'rb') as f:
    model = pickle.load(f)

with open('feature_names.pkl', 'rb') as f:
    feature_names = pickle.load(f)

with open('target_names.pkl', 'rb') as f:
    target_names = pickle.load(f)

print("Models loaded successfully!")

@app.route('/')
def home():
    return render_template('index.html',
                           feature_names=feature_names,
                           target_names=target_names)

@app.route('/api/info',  methods=['GET'])

def get_info():
    """Get information about the API and models"""
    return jsonify({
        'features': feature_names,
        'target_classes': list(target_names),
        'description': 'Iris Flower Classification API'
    })
```

```
@app.route('/api/predict', methods=['POST']) def
predict():
    """Make predictions using the specified model""" try:
        # Get JSON data from request
        data = request.get_json()

        # Validate input
        if not data:
            return jsonify({'error': 'No input data provided'}), 400

        # Extract features and model choice
        features = data.get('features')

        if not features:
            return jsonify({'error': 'Features are required'}), 400

        if len(features) != 4:
            return jsonify({'error': 'Expected 4 features'}), 400

        # Convert to numpy array
        features_array = np.array([features])

        # Make prediction
        prediction = model.predict(features_array)
        prediction_index = int(prediction[0]) predicted_class
        = target_names[prediction_index]

        # Get probability if available
        if hasattr(model, 'predict_proba'):
            probabilities = model.predict_proba(features_array)[0]
            prob_dict = {
                target_names[i]: float(probabilities[i]) for
                i in range(len(target_names))
            }
         else:
            prob_dict = None

        # Return response
        response = {
            'prediction': predicted_class,
            'prediction_index': prediction_index,
            'input_features': {
```

```
                feature_names[i]: features[i]
                for i in range(len(features))


                }
            }

            if prob_dict:
                response['probabilities'] = prob_dict return jsonify(response)

        except Exception as e:
            return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

Step 6: Create Project Structure

```
mkdir templates
```

Step 7: Create templates/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Iris Flower Prediction API</title>
</head>
<body>
    <div class="container">
        <h1>□ Iris Flower Classification API</h1>
        <p class="subtitle">Machine Learning API for predicting iris flower species</p>

        <div class="api-section">
            <h2>□ Dataset Information</h2>
            <p><strong>Features:</strong></p>
            <ul>
                {% for feature in feature_names %}
                <li>{{ feature }}</li>
                {% endfor %}
            </ul>
            <p><strong>Target Classes:</strong></p>
            <ul>
                {% for target in target_names %}
```

```
            <li>{{ target }}</li>
            {% endfor %}
        </ul>
    </div>

    <div class="api-section">
        <h2>⚙ API Endpoints</h2>

        <div class="endpoint">
```

Step 8: Create test_api.py (Testing Script)

```
import requests import json

BASE_URL = 'http://localhost:5000'

def test_info():
    print("Testing /api/info endpoint...") response =
    requests.get(f'{BASE_URL}/api/info')
    print(json.dumps(response.json(), indent=2)) print()

def test_single_prediction():
    print("Testing /api/predict endpoint...") data = {'features': [5.1,
    3.5, 1.4, 0.2]} response = requests.post(
        f'{BASE_URL}/api/predict', json=data,
        headers={'Content-Type': 'application/json'}
    )
    print(json.dumps(response.json(), indent=2)) print()

if __name__ == '__main__':
    print("="*60)
    print("ML API Testing Script")
```

```
    print("="*60) print()

    test_info() test_single_prediction()
```

Step 9: Run the Application

```
python app.py
```

Step 10: Test the API

Open a new terminal and run:

```
pip install requests python
test_api.py
```

Or use cURL:

```
curl -X POST http://localhost:5000/api/predict \
  -H "Content-Type: application/json" \
  -d '{"features": [5.1, 3.5, 1.4, 0.2], "model": "logistic_regression"}'
```

Or open browser at `http://localhost:5000` to see API documentation.

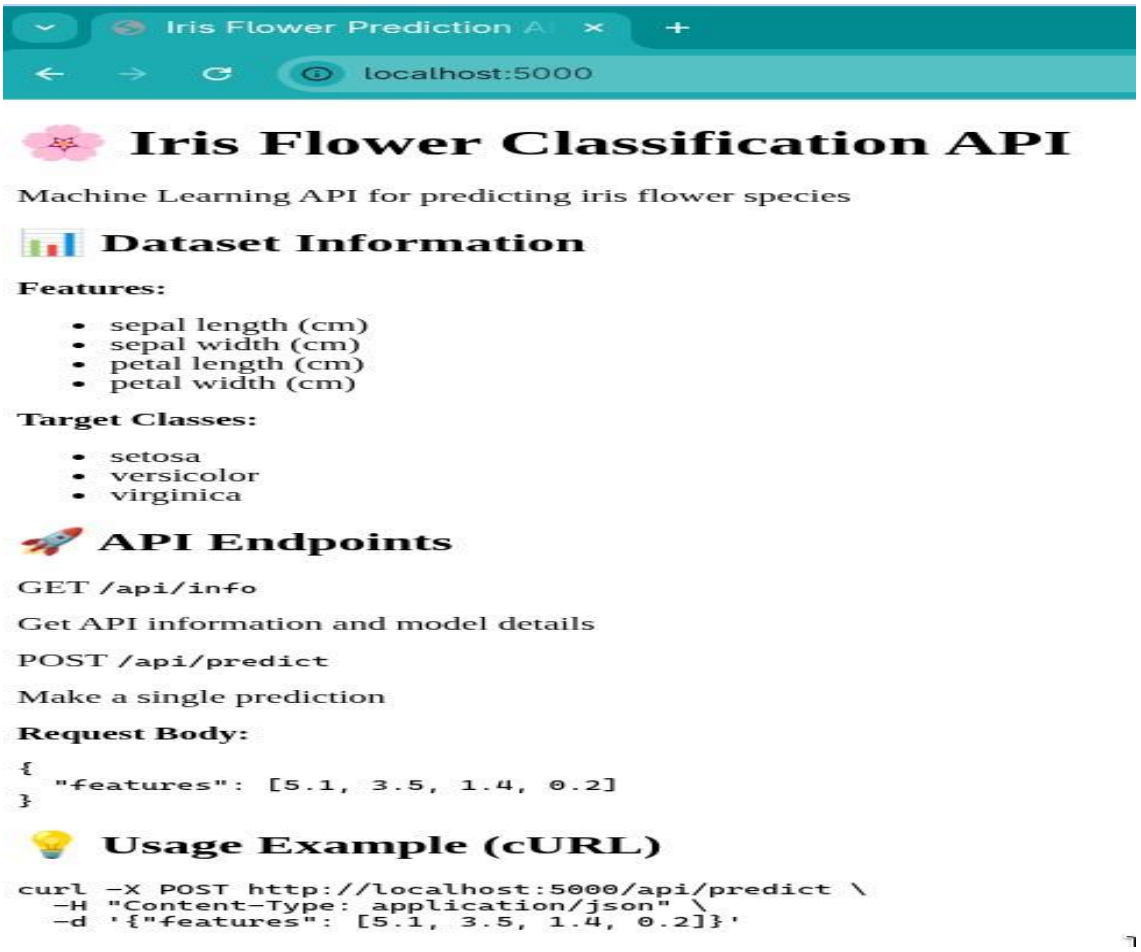
Output

A fully functional REST API that:

- Serves predictions from two ML models
- Returns JSON responses with predictions and probabilities
- Supports single and batch predictions
- Includes API documentation page
- Handles errors gracefully

Screen-shots

Webpage:



Curl request:

```
> curl -X POST http://localhost:5000/api/predict \
        -H "Content-Type: application/json" \
        -d '{"features": [5.1, 3.5, 1.4, 0.2]}'
{
  "input_features": {
    "petal length (cm)": 1.4,
    "petal width (cm)": 0.2,
    "sepal length (cm)": 5.1,
    "sepal width (cm)": 3.5
  },
  "prediction": "setosa",
  "prediction_index": 0,
  "probabilities": {
    "setosa": 0.9783998870533394,
    "versicolor": 0.021600057859436656,
    "virginica": 5.5087223961150795e-08
  }
}
```

Test Execution:

```
personal/lab_manual/exp4_ml_api
 > python test_api.py
============================================================
ML API Testing Script
============================================================

Testing /api/info endpoint...
{
  "description": "Iris Flower Classification API",
  "features": [
    "sepal length (cm)",
    "sepal width (cm)",
    "petal length (cm)",
    "petal width (cm)"
  ],
  "target_classes": [
    "setosa",
    "versicolor",
    "virginica"
  ]
}

Testing /api/predict endpoint...
{
  "input_features": {
    "petal length (cm)": 1.4,
    "petal width (cm)": 0.2,
    "sepal length (cm)": 5.1,
    "sepal width (cm)": 3.5
  },
  "prediction": "setosa",
  "prediction_index": 0,
  "probabilities": {
    "setosa": 0.9783998870533394,
    "versicolor": 0.021600057859436656,
    "virginica": 5.5087223961150795e-08
  }
}
```

Result

Successfully created a production-ready ML prediction API using Flask that loads pre-trained models and serves predictions through RESTful endpoints with proper error handling and documentation.